

Ingegneria del Software – Parte II

L'evoluzione del software



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Prof. Alessandro Saetti

Le leggi di Lehman

Una teoria unificata alla evoluzione del sw



No.	Nome	Descrizione
I 1974	Continuing change	Un grande sistema deve essere continuamente adattato o diventerà progressivamente meno soddisfacente da usare
II 1974	Increasing complexity	Man mano che un sistema evolve la sua complessità aumenta a meno che non si lavori per semplificarne la struttura
III 1974	Self regulation	L'evoluzione di un grande sistema è un processo che si autoregola, ovvero rimangono invariate le proprietà del software, quali il periodo di tempo necessario per la versione successiva, il numero di errori rilevati, ecc.
IV 1974	Organisational stability conservation	Il ritmo di sviluppo di un grande sistema tende a rimanere costante ed è indipendente dalle risorse dedicate allo sviluppo del sistema stesso

Le leggi di Lehman (cont.)

Una teoria unificata alla evoluzione del sw



No.	Nome	Descrizione
V 1978	Conservation of familiarity	La quantità di cambiamenti di ciascuna release tende a rimanere costante (o decrescere)
VI 1991	Continuing growth	Le funzionalità del sistema debbono essere continuamente accresciute durante la vita del software
VII 1996	Declining quality	La qualità del sistema si degrada a meno che il sistema non sia rigorosamente adattato per funzionare nell'ambito lavorativo che va progressivamente modificandosi
VIII 1996	Feedback systems	Il processo evolutivo di un grande sistema richiede feedback su più iterazioni, livelli, e agenti

3

L'evoluzione del software

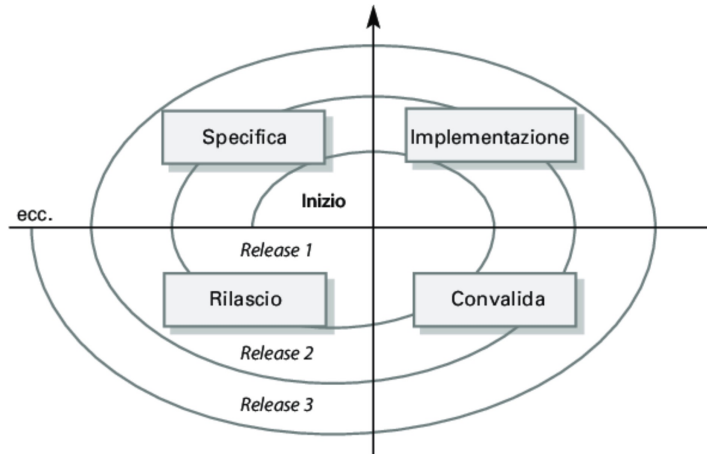
- I grandi sistemi software o le app e i prodotti software di successo hanno di solito lunga vita
- Durante la loro vita i sistemi software devono cambiare se vogliono restare utili
 - sono richiesti nuovi requisiti a causa di cambiamenti nelle aziende e/o nelle aspettative degli utenti
 - è richiesta la correzione di errori
 - è richiesta un adattamento del software per supportare nuovo hardware o software
 - è richiesta un miglioramento delle prestazioni o di soddisfare altre caratteristiche non funzionali



4

Modello a spirale dell'evoluzione

Quando la stessa azienda è responsabile dello sviluppo del software per tutto il suo ciclo di vita...



Prima di internet il rilascio avveniva ogni 2 o 3 anni...
dopo internet il rilascio avviene *ogni poche settimane*

5

Evoluzione e manutenzione del sw

Quando il software personalizzato evolve capita che...

- Il cliente paga una società per sviluppare il software e poi assume la responsabilità dell'evoluzione
- Il cliente stipula un differente contratto con un'altra società per l'evoluzione del software
- Quando la transizione dallo sviluppo all'evoluzione è discontinua il processo di modifica del software dopo la consegna è detto **manutenzione del software**

6

Plan-driven vs agile nella manutenzione

- *Il team di sviluppo ha adottato un processo agile, mentre il team di manutenzione preferisce un approccio basato sui piani*
 - Non c'è una documentazione dettagliata a supporto della fase di manutenzione
 - Può non esserci una chiara e completa definizione dei requisiti
- *Il team di sviluppo ha adottato un processo basato sui piani, mentre il team di manutenzione preferisce un approccio agile*
 - I test di sviluppo potrebbero dovere essere ricostruiti da zero
 - Il codice potrebbe non essere stato rifattorizzato

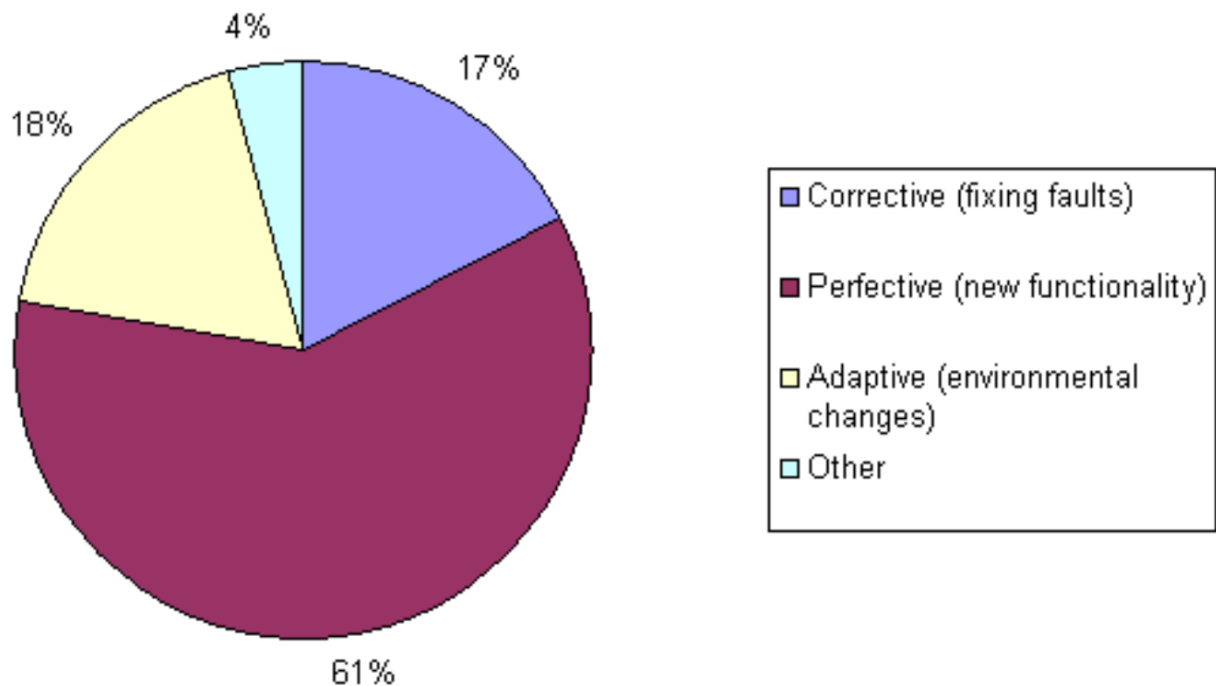
7

Tipi di manutenzione

- Manutenzione **correttiva**
 - Corregge gli errori rimasti dopo il testing pre-delivery
 - Identifica e rimuove i difetti di progettazione e gli errori sui requisiti riscontrati durante l'uso
- Manutenzione **perfettiva**
 - Aggiunge nuove funzionalità su richiesta del cliente
 - Migliora le prestazioni, l'affidabilità, la manutenibilità
- Manutenzione **adattiva**
 - Adatta a un nuovo ambiente hardware, sistema operativo o altro software di supporto
- Manutenzione **preventiva**
 - Cambiamento di qualche aspetto per prevenire guasti
 - Identifica errori prima che vengano riscontrati

8

Costi per tipo di manutenzione



Sarebbe più economico aggiungere le nuove funzionalità durante lo sviluppo. Perché?



9

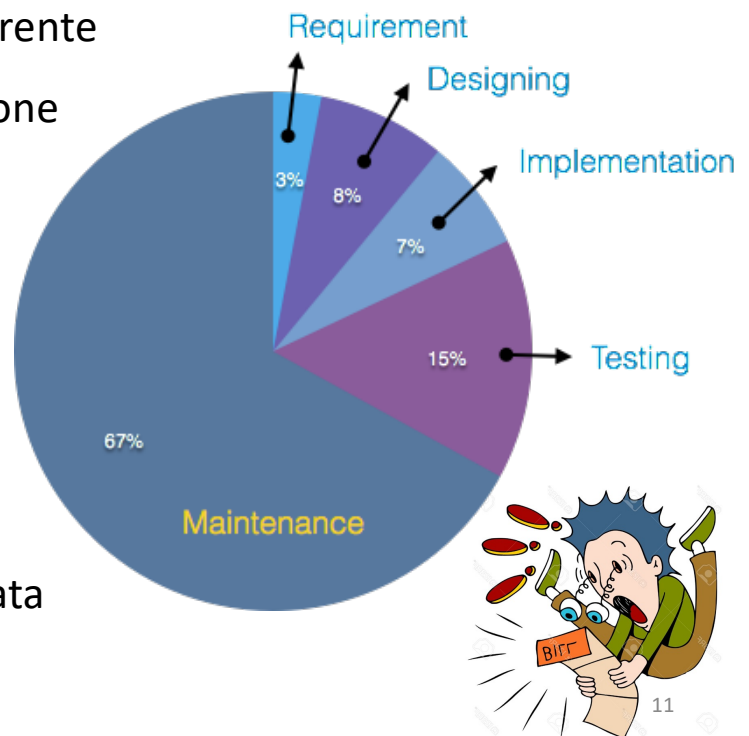
Sistemi ereditati

- Molti vecchi sistemi sono ancora in uso...
- E giocano un ruolo critico nella gestione delle aziende
- **È complicato mantenere un sistema ereditato...**
 - Si basa su tecnologie e linguaggi che non sono più usati
 - La sua struttura è compromessa a causa delle tante modifiche subite
 - È stato scritto per hardware non più disponibili
 - È basato su software di supporto al s.o. non più supportato
 - È composto da programmi applicativi sviluppati in periodi differenti
 - Si sono accumulati immensi volumi di dati (duplicati, incoerenti)
 - I processi aziendali sono stati progettati attorno al sistema ereditato



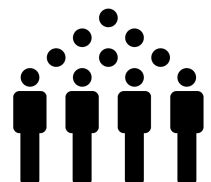
Perché è costoso mantenere i sistemi ereditati?

- Stile del programma incoerente
- Linguaggi di programmazione obsoleti
- Struttura degradata
- Se ottimizzato, è difficile da capire
- Dati in differenti file con strutture incompatibili
- Documentazione inadeguata



Impiegati per sviluppo e manutenzione (solo USA, stime CaperJones)

Year	Development Personnel	Maintenance Personnel	Total Personnel	Maintenance Percent
1950	1,000	100	1,100	9.09%
1955	2,500	250	2,750	9.09%
1960	20,000	2,000	22,000	9.09%
1965	50,000	10,000	60,000	16.67%
1970	125,000	25,000	150,000	16.67%
1975	350,000	75,000	425,000	17.65%
1980	600,000	300,000	900,000	33.33%
1985	750,000	500,000	1,250,000	40.00%
1990	900,000	800,000	1,700,000	47.06%
1995	1,000,000	1,100,000	2,100,000	52.38%
2000	750,000	2,000,000	2,750,000	72.73%
2005	775,000	2,500,000	3,275,000	76.34%
2010	800,000	3,000,000	3,800,000	78.95%
2015	1,000,000	3,500,000	4,500,000	77.78%
2020	1,100,000	3,750,000	4,850,000	77.32%
2025	1,250,000	4,250,000	5,500,000	77.27%



Perché non sostituire i sistemi ereditati?

Troppo costoso e rischioso! 🤔

- I processi aziendali e i modi in cui operano i sistemi ereditati spesso sono intrecciati
- Raramente esiste una specifica dei requisiti completa per un sistema ereditato
- Importanti regole aziendali potrebbero essere state integrate nel software senza essere documentate
- Lo sviluppo di un nuovo sistema è intrinsecamente rischioso

13

Strategia di manutenzione dei sistemi ereditati

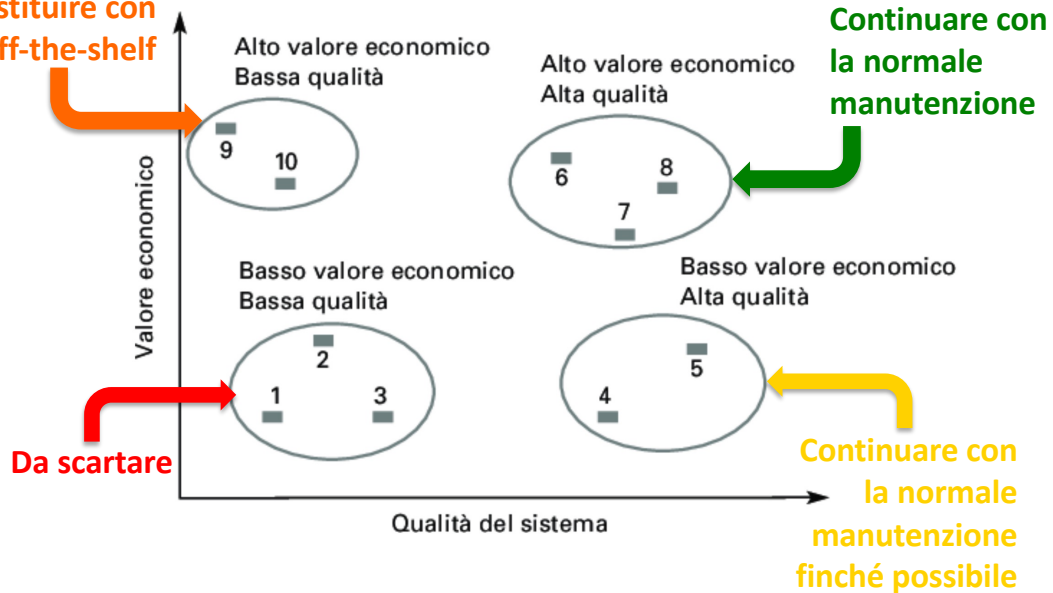
- **Scartare completamente il sistema ereditato**
 - I processi aziendali sono modificati dopo la prima release del sistema e non dipendono più dal sistema ereditato
- **Lasciare il sistema ereditato e continuare con la manutenzione**
 - Il sistema è ancora usato, è in buone condizioni e gli utenti richiedono un numero modesto di modifiche
- **Reingegnerizzare il sistema per migliorarne la manutenibilità**
 - Gli utenti usano ancora il sistema e richiedono sue modifiche ma la qualità del sistema si è deteriorata a causa delle modifiche finora fatte
- **Sostituire una o tutte le parti del sistema con uno nuovo**
 - Quando l'installazione di nuovo hw rende impossibile il funzionamento del sistema ereditato oppure esistono sistemi off-the-shelf

14

Quale strategia scegliere?

- *Prospettiva economica*: l'azienda ha realmente bisogno del sistema?
- *Prospettiva tecnica*: che qualità ha il sw applicativo e l'hw/sw di supporto?

Da reingegnerizzare
o sostituire con
sistemi off-the-shelf



15

Come ricostruire una casa...

1. INVENTORY ANALYSIS

Ispeziona la casa per determinare se è necessario ricostruirla

2. REVERSE ENGINEERING

Prima di ricostruire o ristrutturare la casa assicurati di avere ben capito come era originariamente

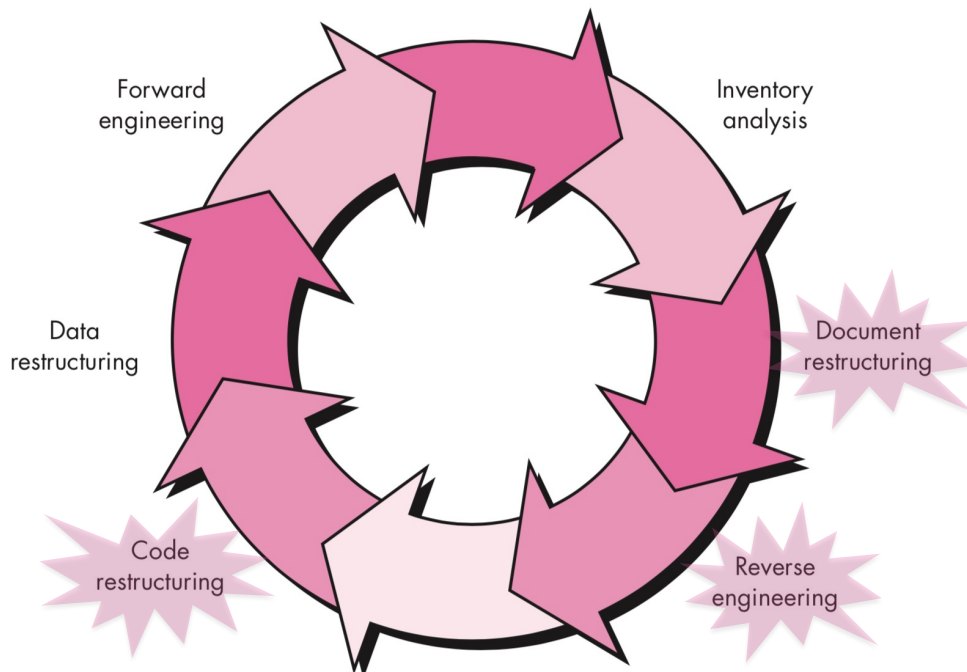
3. RESTRUCTURING/FORWARD ENGINEERING

Per la ricostruzione/ristrutturazione usa solo i più moderni materiali e le buone pratiche per garantire alta qualità



16

Il processo di reingegnerizzazione del sw



17

Document restructuring

La documentazione carente è un problema di molti sistemi ereditati. Come procedere? 🤔

- Lascia perdere se un sistema non richiede frequenti cambiamenti e/o si sta avviando verso la dismissione
- La documentazione ha bisogno di essere aggiornata, ma la tua società ha poche risorse da investire
 - Utilizza un approccio “documenta all’occorrenza”
- Il sistema ha alto valore economico e deve essere completamente ridocumentato

18

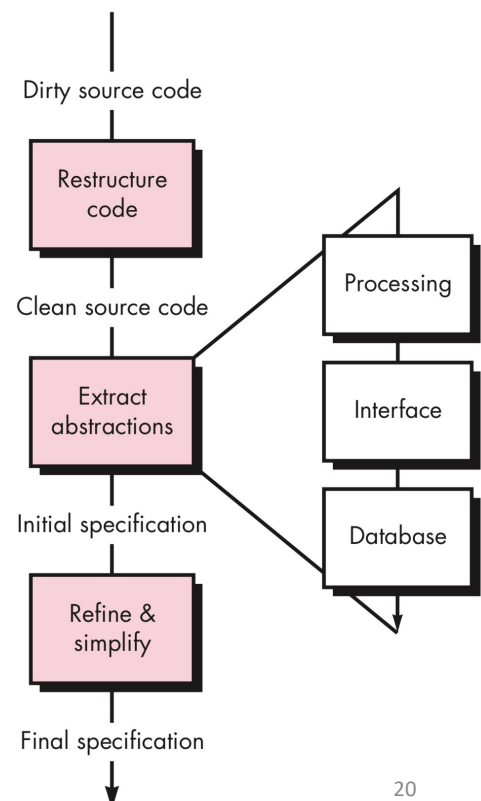
Code restructuring

- L'obiettivo è trasformare un vecchio software in un software meglio strutturato
 - La trasformazione può avvenire entro lo stesso linguaggio di programmazione o meno
 - La trasformazione avviene solitamente entro singoli moduli
 - Simile al refactoring ma non proprio... quali differenze? 🤔
- Strumenti automatici per la ristrutturazione:
 - **DMS Software Reengineering Toolkit**, da Semantic Design
Fornisce tool per COBOL, C/C++, Java, Fortran 90, VHDL
 - **Clone doctor**, sviluppato da Semantic Design
Analizza e trasforma programmi scritti in C, C++, Java, Cobol

19

Reverse engineering

- **Per capire i dati**
 - A livello di componente, significa comprendere le strutture dati interne
 - A livello di sistema, significa comprendere le strutture dati globali (ad es. files, database)
- **Per capire l'elaborazione**
 - Prima si comprende il sistema nel suo complesso
 - Poi si comprende ciascuna componente, considerando generici pattern procedurali
- **Per le interfacce utente**
 - Quali azioni l'interfaccia deve processare?
 - Come risponde il sistema a queste azioni?
 - Che concetto di equivalenza tra GUI usare?



20

Strumenti per il reverse engineering

- **Imagix 4D**

sviluppato da Imagix, San Luis Obispo, CA (www.imagix.com)

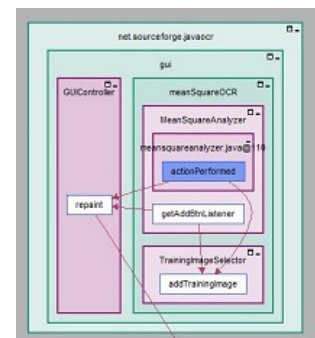
aiuta a comprendere il funzionamento di programmi C e C++, eseguendo sia reverse engineering, sia ridocumentazione del codice sorgente

- **Understand**

sviluppato da Scientific Toolworks, Inc., St. George, UT (www.scitools.com)

effettua reverse engineering, creazione automatica di documenti, calcolo di metriche; aiuta a comprendere, navigare e mantenere codice sorgente di programmi Ada, FORTRAN, C, C++, Java

- Altri strumenti di reverse engineering a:
<https://www.apriorit.com/dev-blog/366-software-reverse-engineering-tools>



Reingegnerizzare o non reingegnerizzare...
questo è il problema!

$$C_{\text{maint}} = (P_3 - (P_1 + P_2)) \times L$$

$$C_{\text{reeing}} = (P_6 - (P_4 + P_5)) \times (L - P_8) - (P_7 \times P_9)$$

$$\text{Costo-beneficio} = C_{\text{reeing}} - C_{\text{maint}} > 0 ?$$

P_1 = current annual maintenance cost for an application

P_2 = current annual operations cost for an application

P_3 = current annual business value of an application

P_4 = predicted annual maintenance cost after reengineering

P_5 = predicted annual operations cost after reengineering

P_6 = predicted annual business value after reengineering

P_7 = estimated reengineering costs

P_8 = estimated reengineering calendar time

P_9 = reengineering risk factor ($P_9 = 1.0$ is nominal)

L = expected life of the system



(Sneed, 1995)

Stimare il costo della manutenzione

La predizione dei costi di manutenzione si basa su:

- **Metriche di prodotto**

- tanto più è complesso un sistema o un componente tanto più è costosa la manutenzione
- esiste evidenza empirica che gran parte dello sforzo speso per la manutenzione si spende su pochi componenti

- **Metriche di processo**

- dopo che un sistema è stato messo in servizio si possono sfruttare i dati sui processi per migliorare le stime di costo

23

Metriche di processo

Alcuni **indicatori di processo** permettono di stimare i costi di manutenzione:

- # richieste di manutenzione correttiva
- # richieste di modifiche importanti
- Tempo medio per l'analisi d'impatto di una modifica
- Tempo medio per soddisfare una richiesta di modifica

➤ Se i valori di queste misure aumentano, abbiamo un'indicazione di manutenibilità decrescente con conseguente aumento dei costi



24

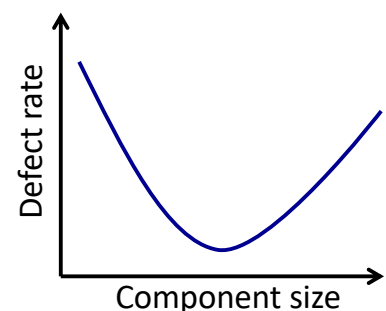
Metriche di prodotto

- **Lines of Code** (LOC o KLOC): misura la dimensione di un componente
- **Complessità ciclomatica**: misura il numero di cammini indipendenti nel flowchart di un componente (o il numero di condizioni binarie)
- **Complessità di Halstead**: misura il numero di operatori e operandi di un componente
- **Metriche strutturali**: misure di accoppiamento basate su Fan in e Fan out dei moduli

25

Lines of code (LOC o KLOC)

- Si contano il numero di linee del programma escludendo i commenti e le linee bianche
- Programmi più grandi richiedono un maggiore sforzo di manutenzione
- Correlata con la densità degli errori (defect rate)
 - Defect rate ha una relazione con la dimensione dei moduli
- **Pro: facile da usare**
- **Contro:**
 - troppo semplicistico
 - fortemente dipendente dal linguaggio



26

Complessità ciclomatica

- Misura il numero di cammini indipendenti nel flow graph del componente
- È il numero di nodi predicato del flow graph + 1
- Più alta è la complessità ciclomatica più complesso è il componente e quindi più rischioso è da modificare

Complessità ciclomatica	Valutazione del rischio
1 ÷ 10	componente semplice, rischio minimo
11 ÷ 20	rischio moderato
21 ÷ 50	componente complesso, rischio alto
Più di 50	componente non testabile, rischio altissimo

27

Complessità di Halstead

Un programma si compone di un numero finito di token, classificati come operandi e operatori

- Operatore: simbolo o parola chiave che specifica un'azione
- Operando: simbolo che rappresenta dati

4 misure	# distinti	# totale
Operatori	n_1	N_1
Operandi	n_2	N_2

Lunghezza: $N = N_1 + N_2$

Vocabolario: $n = n_1 + n_2$

Volume: $V = N \times (\log_2 n)$



➤ Facilmente modificabile se V tra 20 e 1000!



28

Esempio di complessità di Halstead

```
void sort (int a[], int n ) {
    int i, j, t;
    if (n < 2) return;
    for (i = 0; i < n-1; i++) {
        for (j=i+1; j < n; j++ ) {
            if (a[i] > a[j]) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}
```

$V = 80 \times \log_2 (25) \approx 371$
Tra 20 e 1000: OK!

Ignora la definizione della funzione
Conta operatori e operandi

3 < 3 {
5 = 3 }
1 > 1 +
1 - 2 ++
2 , 2 for
9 ; 2 if
4 (1 int
4) 1 return
6 []

1 0
2 1
1 2
6 a
8 i
7 j
3 n
3 t

Misure	# distinti	# totale
Operatori	$n_1 = 17$	$N_1 = 50$
Operandi	$n_2 = 8$	$N_2 = 30$

29

Altre misure di Halstead

Difficoltà. $D = (n_1 / 2) \times (N_2 / n_2)$

- Sorgenti di difficoltà: nuovi operatori e operandi ripetuti
 - Nell'esempio: $17/2 \times 30/8 \approx 32$

Sforzo. $E = V \times D$

- Misura il numero di confronti mentali elementari
 - Ogni secondo la mente umana è capace di eseguire tra 5 e 20 confronti elementari (John Stroud)
 - Usando 18, il tempo per capire/implementare (sec): $T = E / 18$
 - Nell'esempio: $371 \times 32 / 18 \approx 660 \text{ sec} = 11 \text{ min.}$ **Realistico per voi?**



Bugs delivered: $B = E^{2/3} / 3000$

- Misura il numero di bugs nell'implementazione
 - Per C/C++ è risaputo sottostimare
 - Nell'esempio: $B = (371 \times 32)^{2/3} / 3000 = 0.17$

30

Fan in & fan out

Le metriche di struttura modulare stimano l'accoppiamento tra moduli del sistema

- **Fan-in:** # di moduli che chiamano un certo modulo
 - **Fan-out:** # di moduli chiamati da un certo modulo (7 ± 2)
 - I moduli con alto fan-in dovrebbero essere piccoli e appartenere ai livelli inferiori dell'architettura software
 - I moduli con basso fan-in sono di solito grossi e complessi
 - I moduli con alto fan-out dipendono da parecchi moduli e alzano il livello di accoppiamento
- Dunque se ci sono moduli con fan-in piccolo e fan-out grande abbiamo un'indicazione di cattiva qualità di progettazione



31

Opinioni sugli argomenti trattati



- Oppure tramite la pagina web:
<https://forms.gle/JT4ZNe5grJsAARzh9>

Stage o attività di tesi presso Intre

- Sito: www.intre.it
- Intré è una società di consulenza e sviluppo software, specializzata in progetti ad elevata complessità
- Nata nel 1999, conta ad oggi 66 persone, di cui 45 software engineer, 8 UX designer, 3 specialisti in cybersecurity e 3 IT engineer, oltre a personale amministrativo
- Creiamo team misti con persone dei nostri clienti, per lavorare allo sviluppo dei loro prodotti, nei settori più disparati. Ad oggi siamo attivi con 13 team nei sistemi per la gestione delle reti elettriche, gestionali per assicurazioni, sistemi di raccolta di dati statistici, streaming video, sistemi di AI e realtà aumentata per il settore sanitario
- Siamo appassionati di Metodologie Agili, e le applichiamo quotidianamente nel nostro lavoro
- Diamo valore ai nostri clienti sia scrivendo codice, sia facendo mentoring tecnologico e metodologico alle loro persone
- Ognuna delle nostre persone fa in media 25 giornate all'anno di formazione (12%) del tempo lavorativo annuale
- Incentiviamo le persone ad acquisire certificazioni tecniche, sostenendone direttamente i costi
- Partecipiamo assiduamente, sia come speaker, sia come sponsor, sia come semplici partecipanti alle principali conferenze italiane ed europee del settore Software e Metodologie Agili
- Gli studenti interessati possono contattare Alessandro Giardina all'indirizzo alessandro.giardina@intre.it

Domande?

